

Docker

COMS 4156: Advanced Software Engineering
Fall 2024

Presented by Justin Tan (jjt2163)

About Me

- Undergraduate CS major
- Originally from SF , arrived at Columbia in 2021
- 4 years of professional software engineering experience
 - Mostly fintech startups
 - Consumer-facing web applications
 - Distributed ETL pipelines

Docker enthusiasts

My cats playing in containerized snow



Agenda

1. Introduction
2. The Docker ecosystem
3. Using the Docker CLI
4. Live demo (Docker Compose)
5. Kubernetes
6. Closing

Introduction: What is Docker?

Open platform for developing, shipping, and running applications

- **Container-based**
 - Packages software into portable, self-contained units
- **Consistency across environments**
 - Ensures applications run the same everywhere
- **Simplifies development and deployment**
 - Streamlines the process from coding to production



Introduction: What is a container?

Lightweight, self-contained unit for running applications

- **Similar to a lightweight VM**
 - Holds **everything an app needs** to run (code, libraries, dependencies)
- **Efficient and fast**
 - Runs on **shared OS kernel**, unlike a traditional VM
- **Portable and consistent**
 - Runs the same in different environments



Introduction: Containers aren't new

Before Docker, we had:

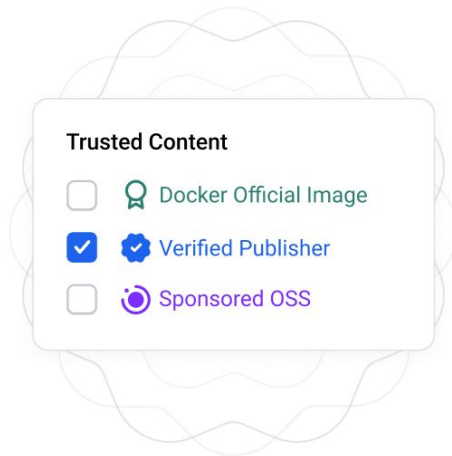
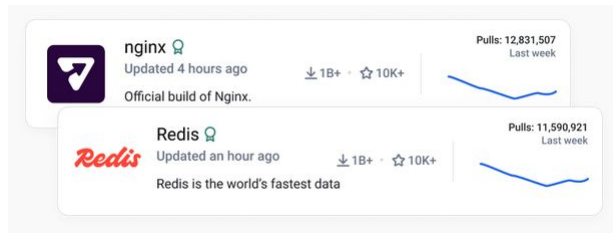
- **chroot (1979)**
 - Basic file system isolation for processes
- **cgroups and namespaces (2007)**
 - Resource control & process isolation in the Linux kernel
- **LXC (2008)**
 - Combined cgroups and namespaces to create the first containerized environments



Introduction: Why Docker is better

Docker improves on these through:

- Ease of use
 - Simple commands and APIs
- Standardized images
 - 14 million images on Docker Hub
- Streamlined isolation
 - Abstracts complexity of resource isolation
 - No need for manual configuration or expertise



Introduction: Who relies on Docker?

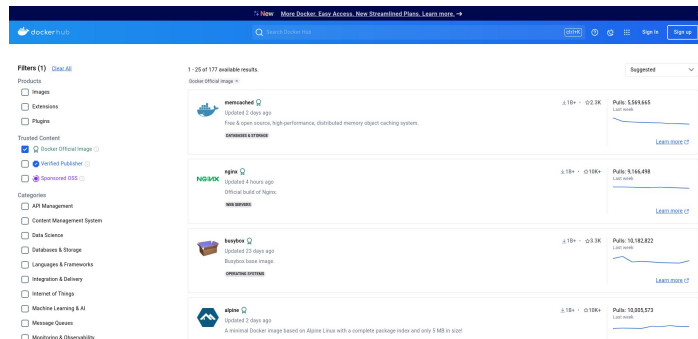
A lot of software teams...



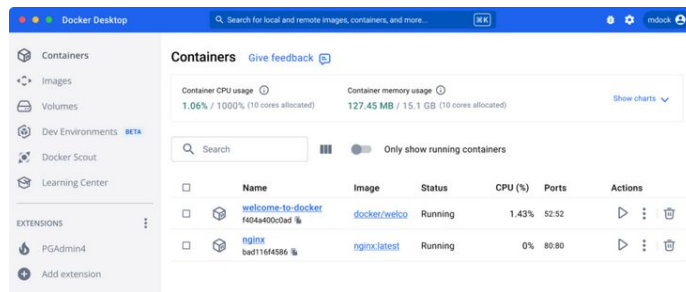
The Docker ecosystem

Docker refers to several components:

- Docker Engine
 - Core runtime that builds and runs containers
- Docker Hub
 - Repository for finding and sharing container images
- Docker CLI and Desktop
 - Tools for managing containers
- Docker Compose
 - Tool for running multi-container apps / services

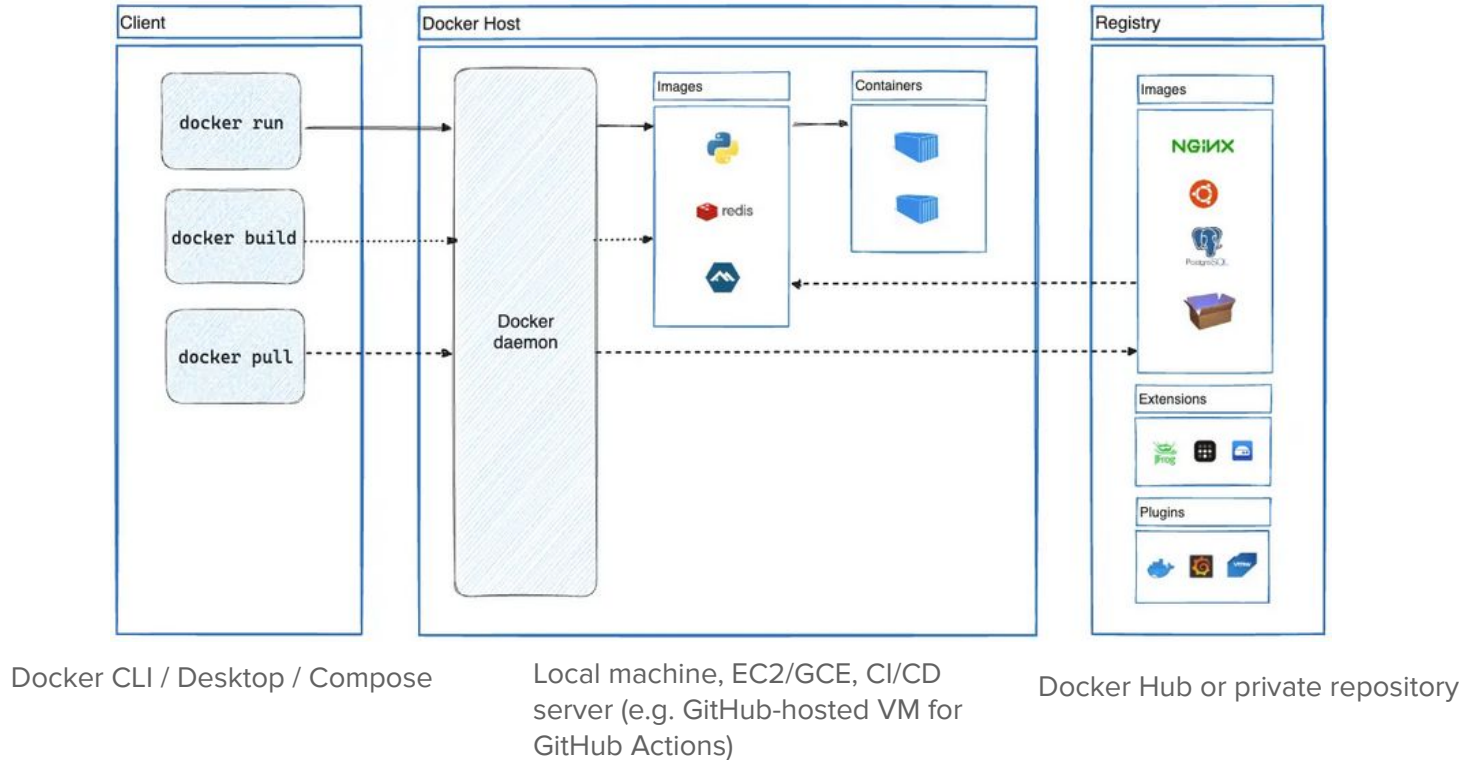


Docker Hub



Docker Desktop

The Docker ecosystem



Using Docker: Docker CLI

Common Docker CLI commands are:

`docker build` – Builds a Docker image from a Dockerfile

`docker run` – Runs a container from a Docker image

`docker ps` – Lists running containers

`docker stop <container ID>` – Stops a running container.

`docker images` – Lists available Docker images on your system.

With these basic commands, you can create, manage, and stop containers.

There is a helpful [cheat sheet](#) available on the Docker website.

Live Demo

We will cover:

- Running a single container (basic Java web server)
- Running multiple containers (Postgres)
- Docker Compose

Kubernetes (k8s)

Docker Compose is sufficient for basic multi-container management

Advantages of k8s:

- **More sophisticated and robust** for production deployments
- Better support for **automation and container orchestration**
- Docker Compose can restart containers, but Kubernetes takes it further...
 - Self-healing (not just for crashes, but also probes for responsiveness)
 - Load balancing
 - Scaling

Closing: Key Takeaways

- containers = consistency
 - Apps run the same way on any machine
 - (assuming the image is built with a CPU-compatible base image)
- Dockerfiles are a blueprint for your app
- Docker Compose simplifies **multi-container app** deployments
 - For example: web server, database, and cache
- Kubernetes (aka k8s)
 - A **separate tool** that goes beyond simple restarts
 - **More resilient** for real-world demands
- Docker and Kubernetes together are **very widely used** in industry

Resources

- [Docker docs](#) (docks??)
 - Docker CLI [cheat sheet](#)
- [Awesome Docker](#)
- [Kubernetes](#)
- [Container Hacks and Fun Images](#), Dockerconf SF '15